COMPUTER METHODS IN APPLIED MECHANICS AND ENGINEERING 61 (1987) 323–338
NORTH-HOLLAND

AN ADAPTIVE FINITE ELEMENT SCHEME FOR TRANSIENT PROBLEMS IN CFD

Rainald LÖHNER
Berkeley Research Associates, Springfield, VA 22150, U.S.A.
and
Laboratory for Computational Physics, Naval Research Laboratory, Washington, D.C. 20375, U.S.A.

Received 25 April 1986
Revised manuscript received 4 August 1986; 21 October 1986

An adaptive finite element scheme for transient problems is presented. The classic *h*-enrichment/coarsening is employed in conjunction with a triangular finite element discretization in two dimensions. A mesh change is performed every *n* timesteps, depending on the Courant number employed and the number of 'protective layers' added ahead of the refined region. In order to simplify the refinement/coarsening logic and to be as fast as possible, only one level of refinement/coarsening is allowed per mesh change. A high degree of vectorizability has been achieved on the CRAY XMP 12 at NRL. Several examples involving shock-shock interactions and the impact of shocks on structures demonstrate the performance of the method, indicating that considerable savings in CPU time and storage can be realized even for strongly unsteady flows.

1. Introduction

The solution of large-scale transient problems around complex geometries is a common problem to many fields of computational fluid dynamics. Among the many applications one can envision we mention the impact of shock waves on structures [1], shock-shock interactions [2], detonations [3], the computation of advancing flame fronts [4], the secondary or tertiary recovery in petroleum engineering [5], and bore formations in estuaries [6]. All of these flow fields share a feature common to all advection-dominated problems: regions of rapid change are embedded in regions where the flow variables vary smoothly. However, the inaccurate representation of these subregions in the numerical solution may influence large regions of the solution domain, which in turn may deteriorate the overall accuracy obtained. As these subdomains, which require a finer gridding, are usually not known a priori and/or change position, either a fine grid has to be employed over the whole domain (which for many problems means an excessive overhead in CPU and storage), or adaptive refinement techniques have to be invoked.

For the reasons just mentioned, adaptive refinement techniques are currently receiving increased attention in the literature [7–20]. It is fair to say that for steady-state problems very robust and elaborate schemes are available (see, for example [13]). As the refinement is performed only as the solution reaches steady state, CPU and storage requirements for the mesh adaptation in this case do not play important roles. This in turn means that one has the

0045-7825/87/$3.50 © 1987, Elsevier Science Publishers B.V. (North-Holland)

freedom to develop more elaborate mesh refinement schemes, e.g. those that refine the grid in only one direction where needed [16].

On the other hand, if transient problems are considered, the following requirements must be met:
- As the grid adaptation has to be performed many times, the adaptation algorithm *must be fast*, and therefore must lend itself to vectorization/parallelization.
- As the grid adaptation process becomes an integral part of any code, the algorithm *should not be storage intensive.*
- As the feature that has been refined may pass again (e.g. a shock reflection), the *original grid should be recovered* after the feature has passed.

This in turn implies that:
- No directional refinement [16] can be used, as these schemes appear as too storage and CPU intensive.
- Classic *h*-enrichment/coarsening must be employed, as it does not require a major storage overhead and due to its simplicity lends itself easily to vectorization.
- Only one level of refinement/coarsening is employed per 'mesh change' in order to minimize the logic involved and thus CPU requirements.
- For triangles, successive subdivision of a triangle into two (see Fig. 1) has to be avoided. This in turn reduces the number of refinement cases considerably.

2. An error indicator

Many possible error indicators have been suggested in the literature (see [7–20] and many other publications), and numerical experience indicates that all perform similarly well. However, in the present context, the following requirements must be met:
- The error indicator must be *fast.*

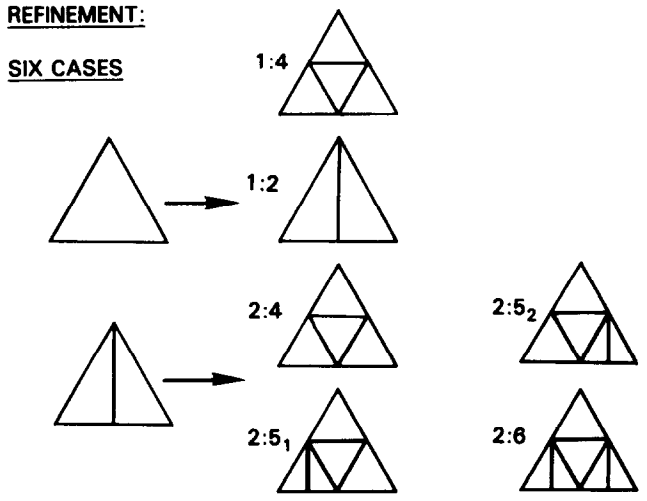


Fig. 1. Allowable refinement cases.

– As the feature may move only very slowly or come to a standstill (e.g. a shock entering a very dense region), the error indicator *must also be reliable for steady-state applications.*
– As systems of equations are solved, and more than one 'key-variable' [11] may be employed, the error indicator should be dimensionless.
– In order to be applicable to a large class of problems, the error estimator should be bounded (independently of the solution), so that preset refinement/coarsening tolerances can be employed.

In order to meet these requirements, a modified form of the classic interpolation estimates [19] used for steady-state computations [14–18, 20] has been adopted. These estimators make use of an appropriate seminorm for the detection of those regions which need further refinement or coarsening, e.g. the $H2$ seminorm [15, 19, 20],

$$\|u - u^h\|_0 \leq c \cdot h^2 \cdot |u|_2 , \tag{1}$$

where u denotes the exact and u^h the approximate solution, c is a mesh-size-independent constant, h is the characteristic mesh size, and

$$|u|_2 = \sqrt{\int_\Omega \sum_{i,j} \left[\frac{\partial^2 u}{\partial x^i \, \partial x^j} \right]^2 \mathrm{d}\Omega} . \tag{2}$$

Second derivatives are justified here because the shape functions used in the finite element discretization are linear. Numerically, we first evaluate the second derivatives at the nodes via a variational statement [22], and then approximate the integral (2) 'conservatively' by taking at element level the maximum second derivative at the nodes. For linear elements of constant length h in 1-D, at the nodes one obtains:

$$e_i = h^{-2} \cdot |U_{i+1} - 2 \cdot U_i + U_{i-1}| . \tag{3}$$

The modified error indicator is given by:

$$E_i = \frac{|U_{i+1} - 2 \cdot U_i + U_{i-1}|}{|U_{i+1} - U_i| + |U_i - U_{i-1}| + \varepsilon[|U_{i+1}| + 2 \cdot |U_i| + |U_{i-1}|]} . \tag{4}$$

We remark the following properties of this modified error indicator:
– By dividing the second derivative by the 'jumps' (gradients) the 'eating-up' effect in the presence of a very strong shock is avoided (i.e. only the value of the normalized $H2$ seminorm is of importance, not the magnitude of the $H2$ seminorm as such).
– Normalizing in this way also has the advantage that the error indicator becomes dimensionless, so that more than one 'key-variable' can be used without encountering dimensioning problems.
– Moreover, the modified error indicator is now bounded ($0 \leq E_i < 1$), so that preset tolerances can be employed (this is of particular importance for transient problems).
– The terms following ε are added as a 'noise' filter in order not to refine 'wiggles' or 'ripples' which may appear due to loss of monotonicity. The value for ε thus depends on the algorithm chosen to solve the PDEs describing the physical process at hand.

The generalization of this error estimator to multidimensional situations is as follows:

$$E^I = \sqrt{\frac{\sum_{k,l} \left(\int_\Omega N^I_{,k} N^J_{,l} \, \mathrm{d}\Omega \cdot U_J \right)^2}{\sum_{k,l} \left(\int_\Omega |N^I_{,k}| [|N^J_{,l} U_J| + \varepsilon (|N^J_{,l}| \, |U_J|)] \, \mathrm{d}\Omega \right)^2}}, \tag{5}$$

where N^I denotes the shape function of node I.

After having determined the values of the error indicators in the elements, all elements lying above a preset threshold value CTORE are refined, while all elements lying below a preset threshold value CTODE are coarsened. Protective layers of elements are added to surround the elements to be refined, so that the 'feature' (e.g. a shock) always travels into an already refined region. The number of protective layers that are added depends on the Courant number employed and the number of timesteps taken between grid modifications. Usually the refinement is performed every 5–10 timesteps, so that for a Courant number of C = 0.2–0.4 one to three protective layers are sufficient.

3. Grid logic

As described above, we limit the number of refinement/coarsening levels per mesh change to one. Moreover, we only allow refinement of a triangle into two or four and avoid the successive refinement of a triangle into two. This implies that there exist only six possible cases for refinement and three for coarsening. These cases are shown in Figs. 1 and 2.

When constructing the algorithm to refine or coarsen the grid one faces the usual decision of speed versus storage. The more information from the previous grid is stored, the faster the new grid may be constructed. As storage requirement minimization was one of the goals of the present research, we tried to keep only the essential information needed between mesh

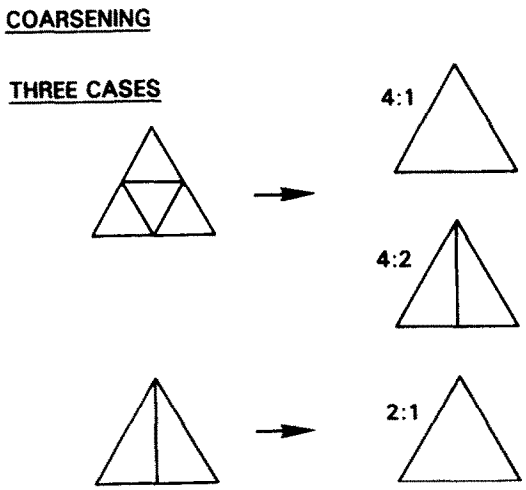


Fig. 2. Allowable coarsening cases.

changes without sacrificing an excessive amount of CPU time. It appears that this 'golden mean' is achieved by keeping six integer locations per element in order to identify the 'parent' and 'son' elements of any element.

The first three integers store the new three neighbor elements ('sons') of an element that has been subdivided into four (the center element of the four is kept as 'parent'). If the element has been subdivided into two elements, the neighbor element is stored in the first integer, whereas the remaining second and third integer locations for this element are set to zero.

In the fourth integer the element from which the present element originated (the 'parent' element) is stored. If the 'parent' element has been subdivided into two, the negative 'parent' element number is stored. This allows to distinguish between the 1:2 and 1:4 refinement cases.

The fifth integer denotes the local side of the 'parent' element this element came from.

Finally, in the sixth integer location the refinement level is remembered. These six integer locations per element are sufficient to construct further refinements or to reconstruct the original grid.

One complete grid change requires algorithmically the following five steps.

Step 1. Construction of the missing grid information needed for a mesh change (basically the sides of the mesh and the sides adjoining each element).
Step 2. Identification of the elements to be refined.
Step 3. Identification of the elements to be deleted.
Step 4. Refinement of the grid where needed.
Step 5. Coarsening of the grid where needed.

3.1. *Step 1*: *Construction of missing grid information*

The missing information consists of the sides of the mesh and the sides adjoining each element. The sides are dynamically stored in two arrays, one containing the two points each side connects and the other one (a pointer array) containing the lowest side-number reaching out of a point. The formation of these two arrays is accomplished in three main loops over the elements, which are partially vectorizable.

After having formed these two side arrays, a further loop over the elements is performed, identifying which sides belong to each element.

3.2. *Step 2*: *Identification of elements to be refined*

The aim of this step is to determine on which sides further gridpoints need to be introduced. To this end, we first determine—using the modified error indicator given by (5) and the prescribed refinement tolerance CTORE—those elements that require further refinement (e.g. LELEM(IE) = 1, otherwise LELEM(IE) = 0).

If protective layers of elements are to be added ahead of the feature to be refined, we perform, for each additional layer, the following set of operations:
- zero an integer point array (e.g. LPOIN(1;NPOIN) = 0);
- loop over the elements to be refined, marking (e.g. LPOIN(IP) = 1) all points connected to these elements;

- zero the integer element array (e.g. LELEM(1;NELEM) = 0);
- loop over all elements; if at least one point of a given element has been marked, refine this element (e.g. LELEM(IE) = 1)

After having identified the elements to be refined, we delete from the list of elements to be refined those elements which are already too small (if a minimum allowed element size has been given), or which have already been refined too many times (if a maximum allowed number of refinement levels has been prescribed).

With the side/element information obtained in Step 1, we can now determine a first set of sides on which new gridpoints need to be introduced. This set of sides is still preliminary, as we don't allow a 1:3 subdivision. Moreover, special logic is incorporated for the 1:2 elements in order to obtain the allowable cases 2:4, $2{:}5_1$, $2{:}5_2$, and 2:6 as shown in Fig. 1. We then perform as many loops as required (usually only one) over the elements, adding further sides whenever a 1:3 case appears. This then yields the final set of sides on which new gridpoints are introduced.

3.3. *Step 3: Identification of elements to be deleted*

As before, we start by determining—using the modified error indicator given by (5) and the prescribed deletion tolerance CTODE—those elements that should be derefined (e.g. LELEM(IE) = −1, otherwise LELEM(IE) = 0).

Thereafter, only the 'parent' elements to be derefined are considered further. For these elements, a check is performed whether their respective 'son' elements should also be coarsened (e.g. LELEM(IESON) = −1). Only those 'parent' elements are kept, for which all 'son' elements have been marked for derefinement. These are marked accordingly (e.g. LELEM(IE) = −2).

Although we are now left with an 'inner core' of elements to be coarsened, the 4:3 coarsening has to be avoided (see Fig. 2). This is accomplished as follows:

- zero an arbitrary point array (e.g. LPOIN(IP) = 0);
- for each 'inner-core parent element' (LELEM(IE) = −2), add one to the LPOIN array for the points corresponding to this element (e.g. LPOIN(IP) = LPOIN(IP) + 1);
- we refer to all points with LPOIN(IP) = 2 as 'total deletion points';
- only consider for coarsening those 'inner-core parent elements' (LELEM(IE) = −2) which have more than one 'total deletion point' (LPOIN(IP) = 2), marking them (e.g. LELEM(IE) = −3).

It is an easy exercise to verify that by using this algorithm the 4:3 coarsening pattern is avoided. A second re-marking of LPOIN as before for the LELEM(IE) = −3 elements then yields the final set of 'total deletion points' (LPOIN(IP) = 2).

3.4. *Step 4: Refinement of the grid where needed*

The introduction of further points and elements is performed in two independent steps, which in principle could be performed in parallel.

To add further points, the sides marked for refinement in Step 2 are grouped together. For each of these sides a new grid point will be introduced. The interpolation of the coordinates and unknowns is then performed using the side/point information obtained in Step 1. These new coordinates and unknowns are added to their respective arrays. In the same way new boundary conditions are introduced where required.

In order to add further elements, the sides marked for refinement are labelled with their new grid-point number. Thereafter, the element/side information obtained in Step 1 above is employed to add the new elements. The elements to be refined are grouped together according to refinement case, see Fig. 1. Each case is treated in block fashion (in a separate subroutine). For the 1:4 subdivision, we keep the 'center element' of the refined group as the 'parent element', and add the remaining three elements to the end of the list. For the 1:2 subdivision, we keep the 'left' element as the 'parent element'. The other refinement cases follow accordingly.

3.5. Step 5: Coarsening of the grid where needed

The deletion of points and elements is again performed in two independent steps, which in principle could be performed in parallel.

The points to be deleted having been marked in Step 3 above, all that remains to be done is to fill up the voids in the coordinate, unknown, and boundary condition arrays by renumbering points and boundary conditions.

The deletion of elements is again performed blockwise, by grouping together all elements corresponding to a particular derefinement case (see Fig. 2). Thereafter, the elements are also renumbered (in order to fill up the gaps left by the deleted elements), and the point renumbering is taken into consideration within the connectivity arrays.

We conclude the description of the grid logic algorithm with the following remarks:
- The grid logic does not employ a classical tree structure.
- The whole mesh refinement/coarsening process described can be vectorized to a large extent, particularly if the machine available vectorizes GATHER/SCATTER loops efficiently. On the CRAY XMP 12 at NRL the present algorithm performs more than 99% of all operations necessary for a mesh change inside long (≥ 50) vectorizable loops. In fact, the CPU time spent in nonvectorizable loops is of the order of 1%, indicating a high degree of efficiency.
- The processing rate for the grid adaptation alone is of the order of 100 μs per grid point per mesh change. Although this figure seems high, one has to bear in mind that the adaptation is performed only every 5–10 timesteps, which means that effectively it is much lower.

4. Numerical examples

We confine the numerical examples to the computation of highly unsteady compressible flows with shock waves. Other fields of applications are obvious. As the basic hydrodynamics solver we employ the FEM-FCT code of Löhner et al. [21], which is capable of reproducing moving and stationary shocks over two elements without loss of monotonicity. For this class of problems and the algorithm employed it was found that the following choice of refinement/coarsening parameters produced acceptable results:
- refinement tolerance: CTORE = 0.3;
- coarsening tolerance: CTODE = 0.1;
- noise parameter: $\varepsilon = 0.2$;
- key variable: density.

We also tested other key variables as error indicators, but it was found that for the class of problems under consideration, i.e. compressible Euler equations, the density gave the best

results (pressure is not a good error indicator at contract discontinuities).

Unless otherwise stated this set of parameters was employed for all the examples shown below.

EXAMPLE 4.1. Circular blast wave. The problem statement, as well as the solutions obtained are shown in Fig. 3. A quadrant of a cylinder in the lower left-hand corner was given a density of 10.0 and a pressure of 40.0, while the rest of the computational region was filled with density 1.0 and pressure 1.0. Because all grid points inside radius 5.1 were disturbed and all gridpoints outside were not, the surface of the cylinder on the finite element grid is not completely circular. The number of refinement levels allowed in this case was NREMX = 5, which would correspond to a regular grid of $160 * 160 = 25600$ points, and the grid was modified every 10 timesteps with two protective layers, while the hydrodynamics code was run at a Courant number of C = 0.2. This case was run to test the symmetry or 'circularity' of the numerical solution. Although the numerical solution obtained is not completely symmetric

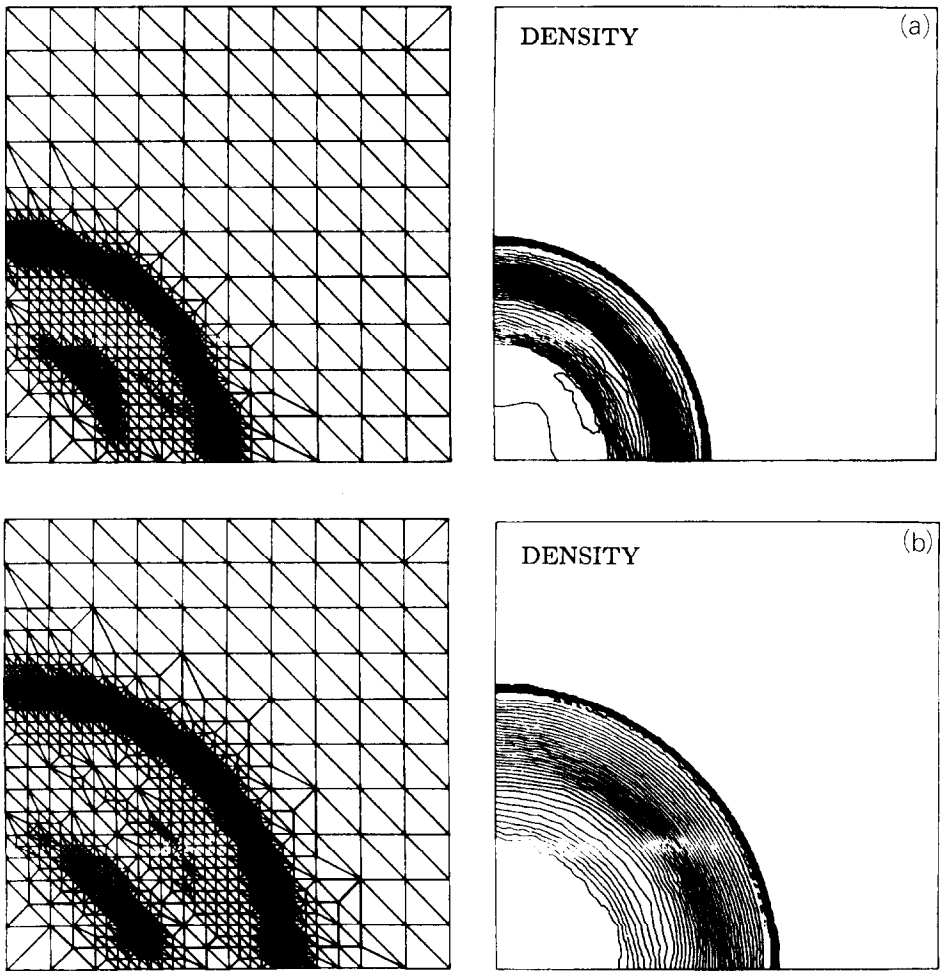


Fig. 3. Blast-wave propagation. (a) $T = 5.13$, NELEM = 3132, NPOIN = 1599. (b) $T = 7.68$, NELEM = 4459, NPOIN = 2267.

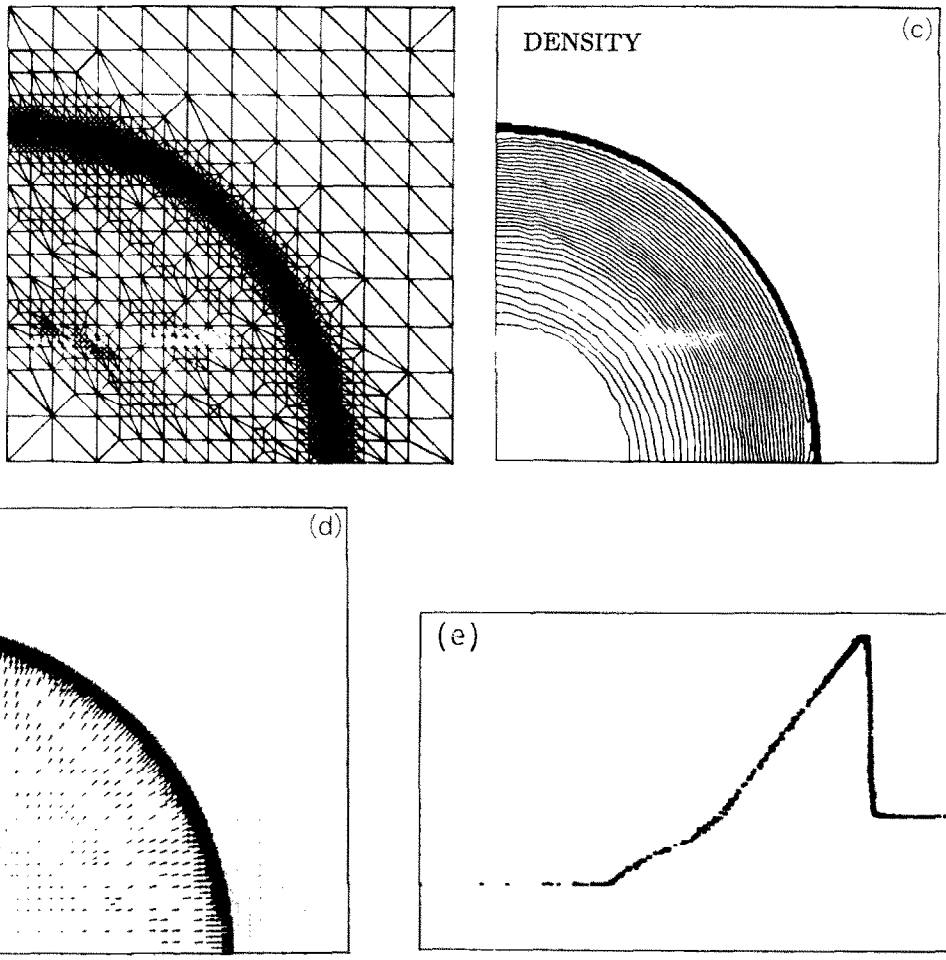


Fig. 3 (cont.). Blast-wave propagation. (c) $T = 9.3$, NELEM = 4508, NPOIN = 2290. (d) Velocity field. (e) Plot of the density versus R for all gridpoints.

with respect to the 45° line (the symmetry is lost after fifteen mesh changes), the difference between the residuals of the x- and y-momenta is only discernable in the fourth significant digit.

The evolution of the main shock is clearly visible in Figs. 3(a), (b), and the solution at time $T = 9.3$ is shown in Figs. 3(c), (d). Observe that at $T = 9.3$, the number of gridpoints needed (2290) when adapting the mesh as compared to a uniformly refined grid (25 600) is more than a factor of 11. Admittedly, this is a simple example, but it clearly demonstrates the great potential that adaptive refinement offers for a large class of problems. The density for all points in the domain is shown in Fig. 3(e) plotted versus the radial distance from the origin, indicating a nearly perfect symmetry.

EXAMPLE 4.2. Shock impinging on a half cylinder. The problem under consideration is shown in Fig. 4(a). A strong shock ($M_s = 10$) runs onto a half cylinder, is reflected in part,

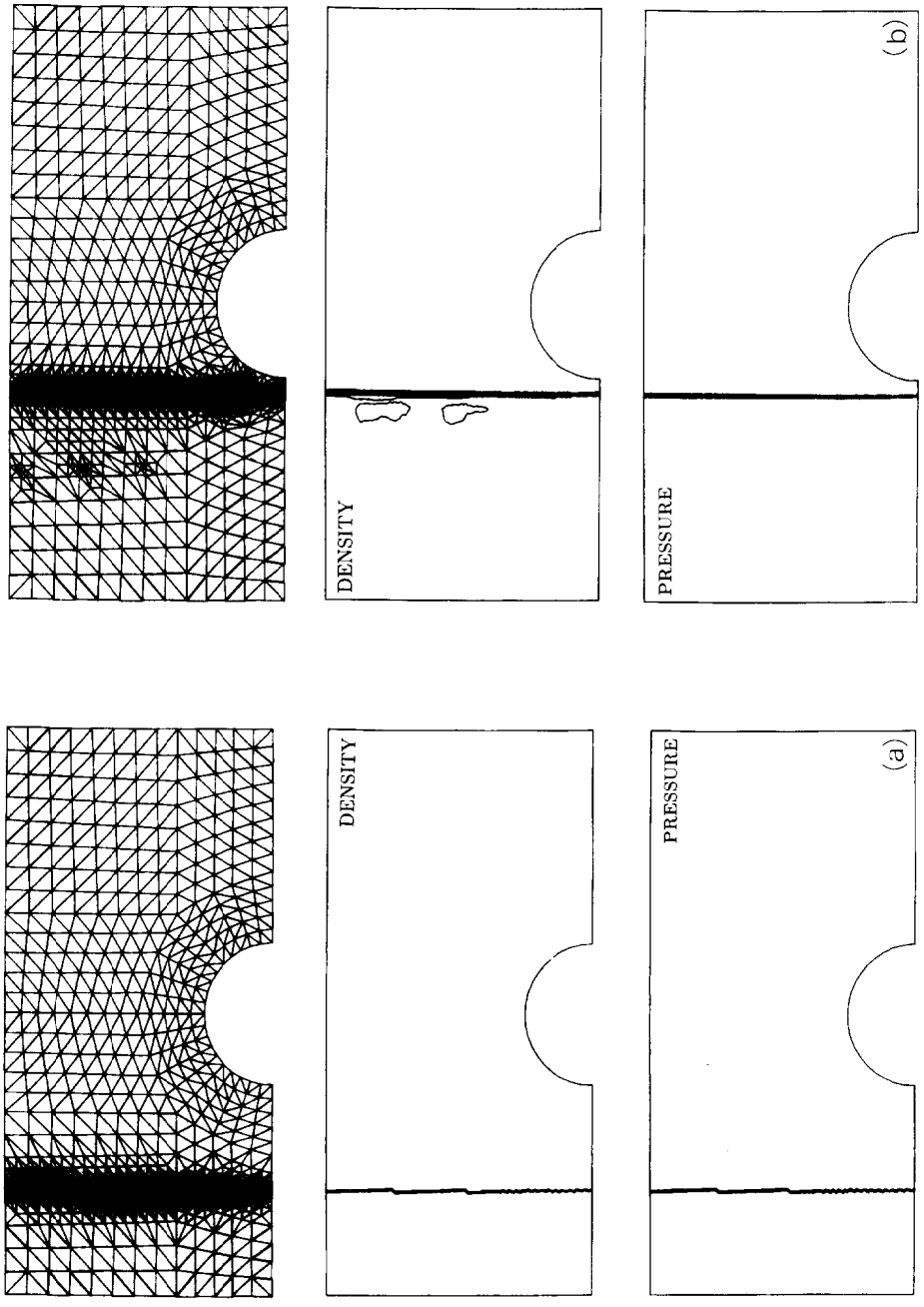


Fig. 4. Shock impinging on a half cylinder. (a) $T = 0.0$, NELEM = 2554, NPOIN = 1335. (b) $T = 0.1$, NELEM = 3096, NPOIN = 1608.

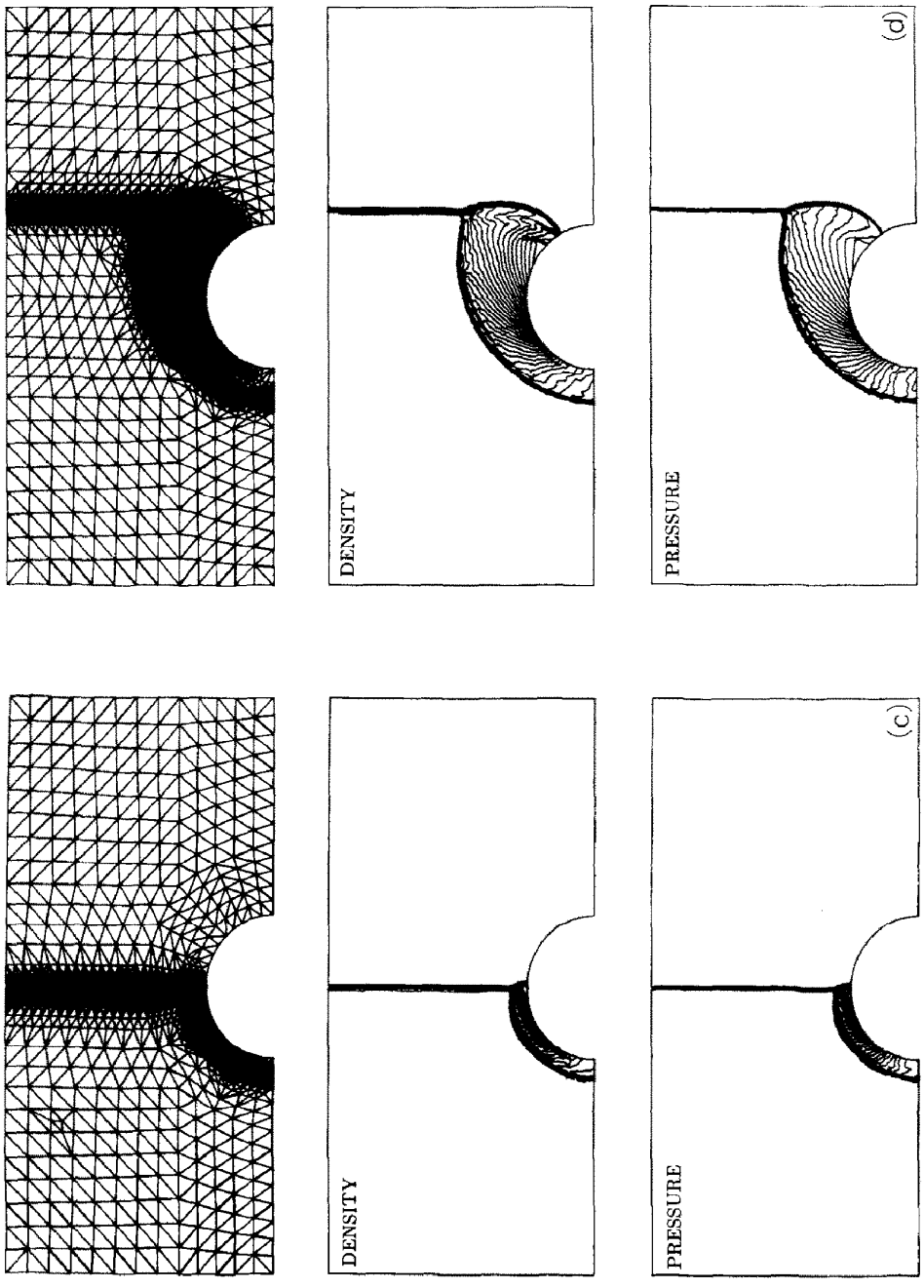


Fig. 4 (cont.). Shock impinging on a half cylinder. (c) $T = 0.2$, NELEM = 4628, NPOIN = 2394. (d) $T = 0.3$, NELEM = 9057, NPOIN = 4626.

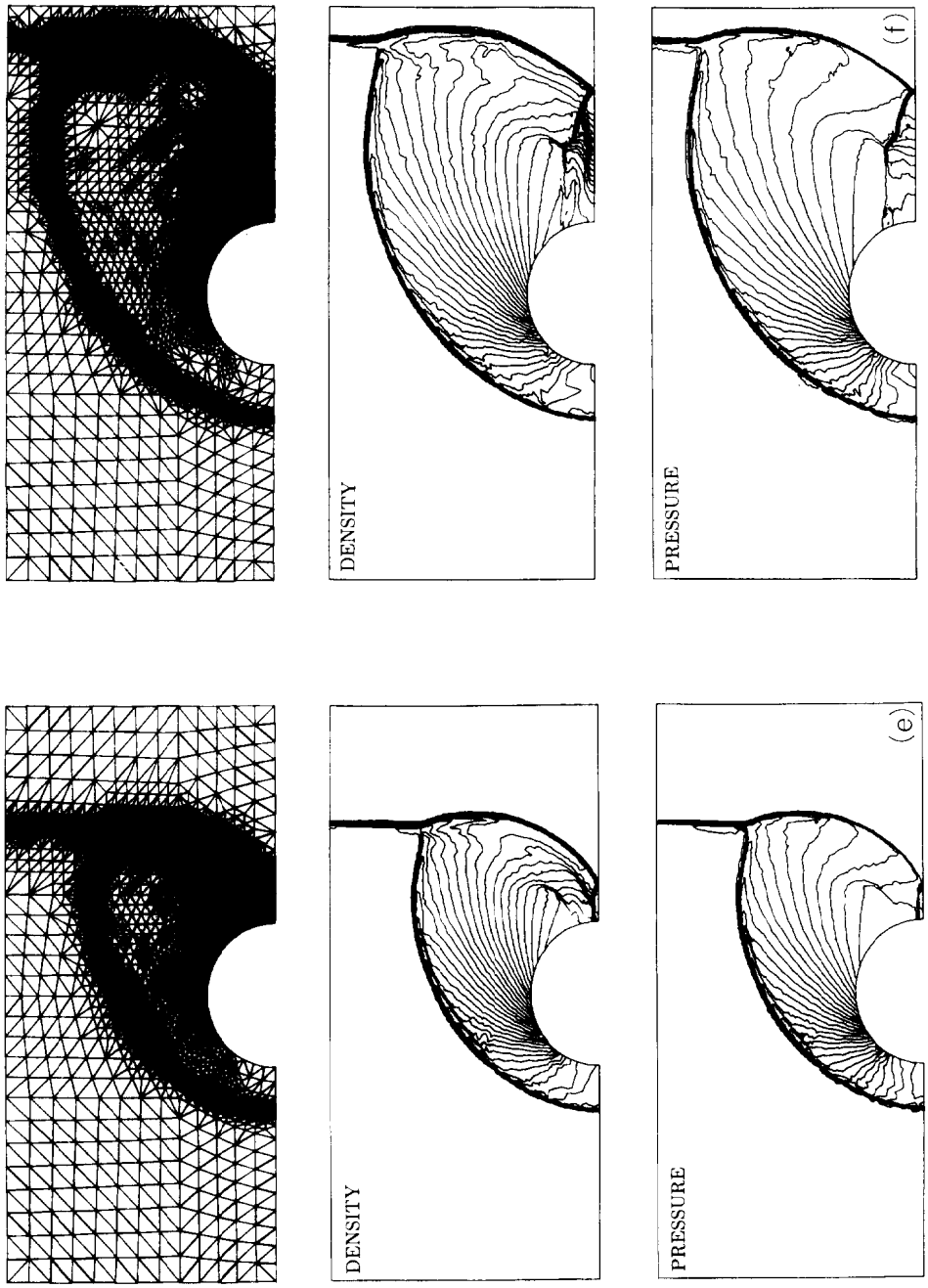


Fig. 4 (cont.). Shock impinging on a half cylinder. (e) $T = 0.4$, NELEM = 10 071, NPOIN = 5142. (f) $T = 0.5$, NELEM = 12 020, NPOIN = 6129.

forms a Mach stem and produces an expansion behind the cylinder. The number of refinement levels allowed in this case was NREMX = 3 (which would correspond on a uniformly refined grid to NPOIN = 28 032 points), and the grid was modified every 5 timesteps with only one protective layer, while the hydrodynamics code was again run at a Courant number of C = 0.2.

The solutions obtained at different times during the transient are depicted in Figs. 4(a)–(f). The maximum number of gridpoints needed for this computation was NPOIN = 6129, which implies a savings factor of more than four over a uniformly refined grid.

EXAMPLE 4.3. Shock impinging on two obstacles. This problem is essentially the same as the previous one, only the geometry is considerably more involved. It is reproduced here in order to demonstrate not only the effectiveness of the adaptive grid scheme presented, but also of unstructured grids in general when realistic problems need to be simulated. The two obstacles, as well as the solution at time $T = 0.0$ (a strong shock, $M_s = 10$, coming from the left) are shown in Fig. 5(a). The number of refinement levels allowed in this case was NREMX = 3. Had the grid been refined uniformly, this would correspond to NPOIN = 38 080 points. The grid was modified every 5 timesteps with one protective layer. The Courant number was again set to C = 0.2.

The solutions obtained at different stages during the transient are depicted in Figs. 5(a)–(d). Observe the amount of detail that the adaptive refinement procedure is capable of reproducing. The maximum number of points required (corresponding to time $T = 0.8$) was NPOIN = 8101, again less than four times what uniform refinement would have required. During most of the computation, the number of grid points is of course much lower (the savings much bigger). As the physics become more involved, more grid points are required, and correspondingly larger portions of the domain are refined.

5. Conclusions

An adaptive finite element scheme for transient problems has been presented. The classic h-enrichment/coarsening is employed in conjunction with a triangular finite element discretization in two dimensions. The grid is adapted every n timesteps, depending on the Courant number employed and the number of 'protective layers' added ahead of the refined region. Particular emphasis was placed on speed and low storage requirements from the outset. Therefore, only one level of refinement/coarsening was allowed per mesh change, and the subsequent subdivision of a triangle into two was avoided. It has been demonstrated that with these restrictions in mind a high degree of vectorizability can be achieved on modern supercomputers.

Numerical examples taken from practical shock-shock and shock-structure interaction problems indicate that with the present approach saving factors in both CPU and storage requirements of more than an order of magnitude as compared to uniform refinement are attainable, without deteriorating the accuracy of the solutions.

An obvious extension of the present work is the development of a similar scheme in three dimensions involving tetrahedrons, and the combination of different types of elements (quads, triangles) in one single code.

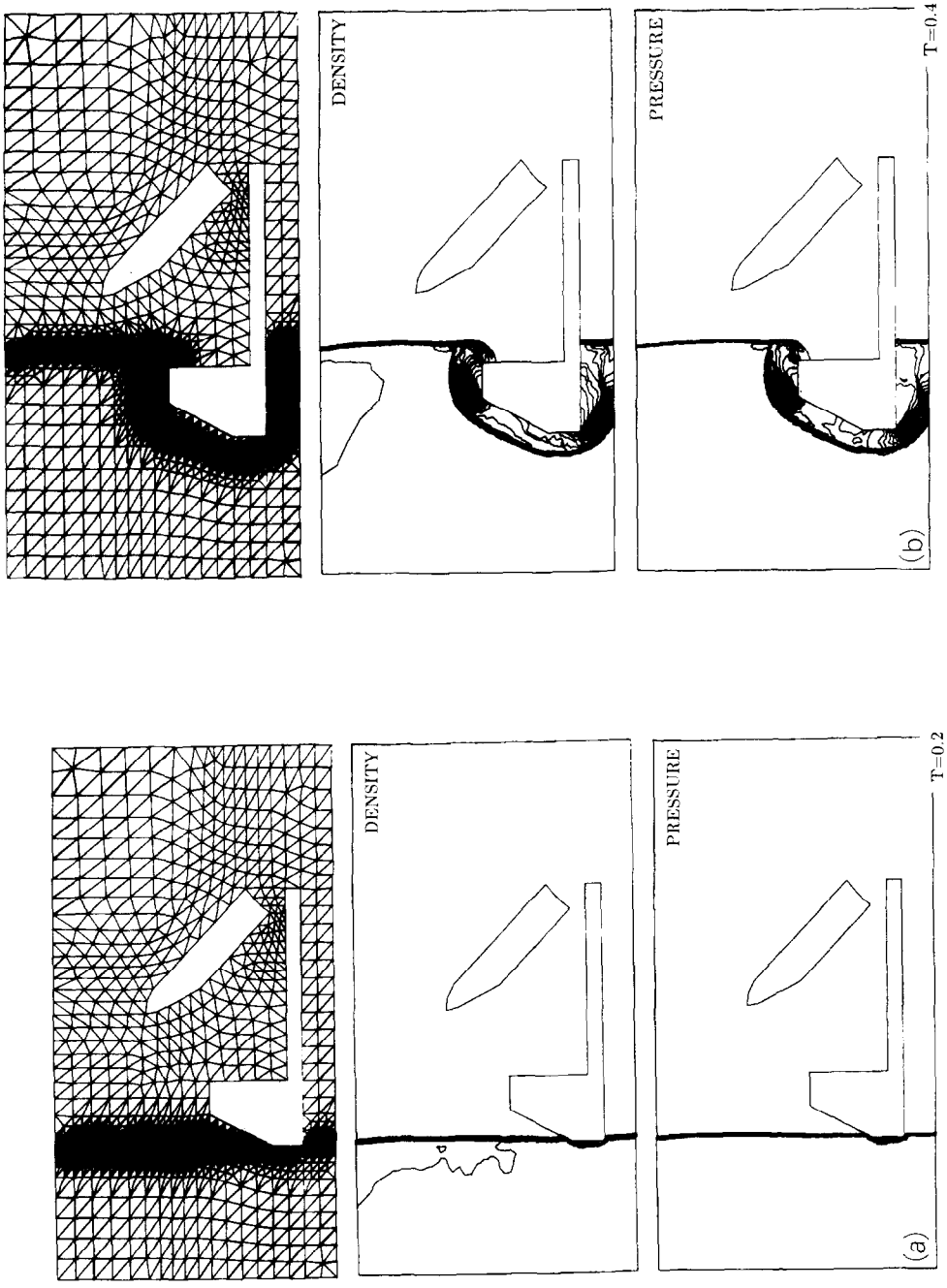


Fig. 5. Shock impinging on two obstacles. (a) $T = 0.2$, NELEM = 3545, NPOIN = 1834. (b) $T = 0.4$, NELEM = 6038, NPOIN = 3153.

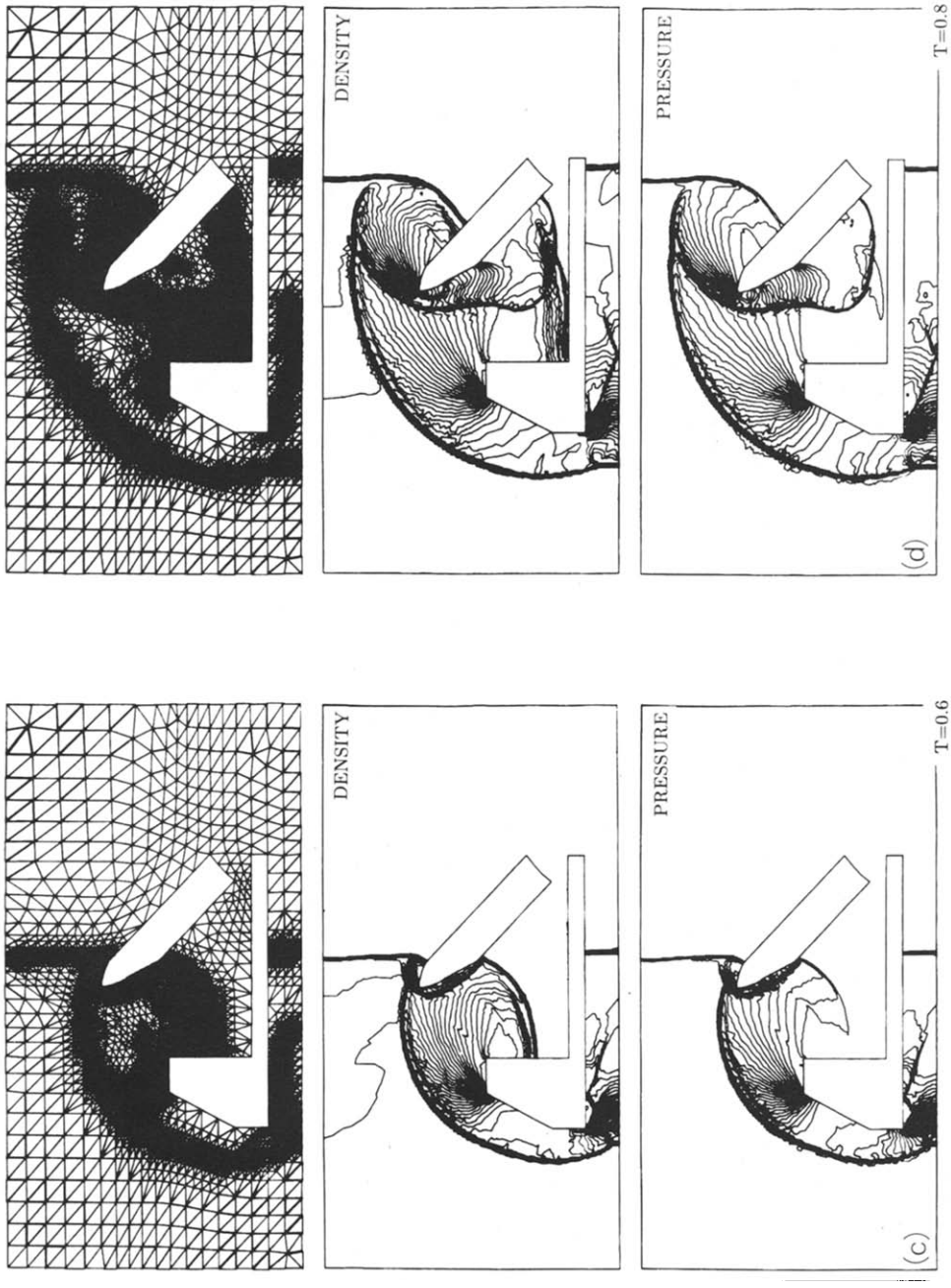


Fig. 5 (cont.). Shock impinging on two obstacles. (c) $T = 0.6$, NELEM = 10 713, NPOIN = 5526. (d) $T = 0.8$, NELEM = 15 798, NPOIN = 8101.

Acknowledgment

It is a pleasure to acknowledge fruitful discussions with Drs. K. Morgan, M. Fritts and J.P. Boris during the development of the schemes presented.

References

[1] H.M. Glaz, P. Colella, I.I. Glass and R.L. Deschambault, A numerical study of oblique shock-wave reflections with experimental comparisons, Proc. Roy. Soc. London Ser. A 398 (1985) 117–140.
[2] M. Fry, J. Tittsworth, A. Kuhl, D.L. Book, J.P. Boris and M. Picone, Transport algorithms with adaptive gridding, in: C.E. Treanor and J.G. Hall, eds., Proceedings 13th International Symposium on Shock Tubes and Waves (1982) 376–384.
[3] K. Kailasanath, E.S. Oran, J.P. Boris and T.R. Young, Determination of detonation cell size and the role of transverse waves in two-dimensional detonations, Combustion and Flame 61 (1985) 199–209.
[4] N. Peters and J. Warnatz, eds., Numerical Methods in Laminar Flame Propagation, Vieweg Notes on Numerical Fluid Mechanics Vol. 6 (Vieweg, Braunschweig, 1982).
[5] R.E. Ewing, ed., Special Issue on Oil Reservoir Simulation, Comput. Meths. Appl. Mech. Engrg. 47 (1984).
[6] O.C. Zienkiewicz, private communication.
[7] P.A. Gnoffo, A finite-volume, adaptive grid algorithm applied to planetary entry flowfields, AIAA J 21 (1983) 1249–1254.
[8] K. Nakahashi and G.S. Deiwert, A three-dimensional adaptive grid method, AIAA-85-0486, 1985.
[9] M.J. Berger and J. Oliger, Adaptive mesh refinement for hyperbolic partial differential equations, J. Comput. Phys. 53 (1984) 484–512.
[10] M.D. Smooke and M.L. Koszykowsky, Two-dimensional fully adaptive solutions of solid-solid alloying reactions, J. Comput. Phys. 62 (1986) 1–25.
[11] J.F. Dannenhoffer and J.R. Baron, Adaptive procedure for steady state solution of hyperbolic equations, AIAA-84-0005, 1984.
[12] J.F. Dannenhoffer and J.R. Baron, Grid adaptation for the 2-D Euler equations, AIAA-85-0484, 1985.
[13] J.F. Dannenhoffer and J.R. Baron, Robust grid adaptation for complex transonic flows, AIAA-86-0495, 1986.
[14] R. Löhner, K. Morgan and O.C. Zienkiewicz, An adaptive finite element procedure for high speed flows, Comput. Meths. Appl. Mech. Engrg. 51 (1985) 441–465.
[15] R. Löhner, K. Morgan and O.C. Zienkiewicz, Adaptive grid refinement for the compressible Euler equations, in: I. Babuška et al., eds., Accuracy Estimates and Adaptive Refinements in Finite Element Computations (Wiley, New York, 1986) Ch. 15, 281–297.
[16] R. Löhner and K. Morgan, Improved adaptive refinement strategies for finite element aerodynamic computations, AIAA-86-499, 1986.
[17] B. Palmerio, V. Billey, A. Dervieux and J. Periaux, Self-adaptive mesh refinements and finite element methods for solving the Euler equations, in: Proceedings ICFD Conference on Numerical Methods for Fluid Dynamics, Reading, United Kingdom, 1985.
[18] F. Angrand, V. Billey, A. Dervieux, J. Periaux, C. Pouletty and B. Stoufflet, 2-D and 3-D Euler flow calculations with a second-order accurate Galerkin finite element method, AIAA-85-1706, 1985.
[19] J.T. Oden, Notes on grid optimization and adaptive methods for finite element methods, TICOM Rept., University of Texas at Austin, Austin, TX, 1983.
[20] J.T. Oden, P. Devloo and T. Strouboulis, Adaptive finite element methods for the analysis of inviscid compressible flow: Part I. Fast refinement/unrefinement and moving mesh methods for unstructured meshes, Comput. Meths. Appl. Mech. Engrg. 59 (1986) 327–362.
[21] R. Löhner, K. Morgan, M. Vahdati, J.P. Boris and D.L. Book, FEM-FCT; Combining high resolution with unstructured grids, J. Comput. Phys. (1986) to appear.
[22] O.C. Zienkiewicz and K. Morgan, Finite Elements and Approximation (Wiley, New York, 1983).